Molly Pate

3 May 2026

Final Project Report

Introduction

K-means clustering is a widely used unsupervised learning algorithm that partitions data into groups by iteratively assigning points to the nearest centroid and recomputing centroid positions. Although simple, it becomes computationally expensive on large, high-dimensional datasets due to the large number of distance calculations required between data points and cluster centers.

This structure makes K-means well suited for GPU acceleration. GPUs are designed for massively parallel computation and can execute thousands of threads simultaneously, making them effective for workloads with many independent operations. In K-means, distance computations across data points and centroids can be parallelized, offering the potential for significant speedup over CPU implementations.

This project compares CPU and CUDA-based GPU implementations of K-means, focusing on how different optimization strategies impact performance and scalability. The GPU version leverages parallel thread execution, shared memory, and memory coalescing to reduce memory overhead, while the CPU implementation serves as a sequential baseline.

Key optimizations explored include shared memory tiling, efficient global memory access patterns, and parallel reduction techniques for updating centroids. The project also considers algorithmic variations such as mini-batch K-means to evaluate trade-offs between performance.

Overall, this work aims to quantify the performance benefits of GPU acceleration for K-means and identify which architectural and algorithmic optimizations most strongly influence efficiency.

Algorithm

For this project, two algorithmic implementations were used. Initially, the work focused on a direct comparison of the standard (batch) K-means clustering algorithm on CPU versus GPU. This baseline version uses the full dataset in each iteration and provides a clear view of the performance differences introduced by GPU parallelization. As the project progressed, the mini-batch K-means stategy was explored to address the high computational and memory costs associated with large datasets for the specific CIFAR-10 dataset application. This approach reduces workload per iteration by operating small, randomly sampled subsets of the data, allowing for improved scalability while maintaining comparable clustering quality.

2.1 Standard (Batch) K-Means Algorithm

The standard K-means algorithm follows an iterative refinement process in which the full dataset is used at every iteration. The implementation begins by initializing K centroids, either randomly or using a heuristic such as k-means++ (for the sake of focusing on the optimizations of this project, and knowing CIFAR-10 has 10 clusters, this was hard-coded in the implementation). During each iteration, the assignment step computes the distance between every data point and all centroids, assigning each point to the nearest cluster based on Euclidean distance. In the GPU version, this step is parallelized such that each thread is responsible for computing distances for a single data point, significantly accelerating this computation-heavy stage. After all points are assigned, the update step recomputes each centroid by averaging all points belonging to that cluster. This requires a reduction operation over assigned points, which in the GPU implementation is handled using parallel accumulation and synchronization strategies. The algorithm repeats these steps until convergence, defined as minimal centroid movement or reaching a maximum number of iterations.

A key bottleneck in this implementation comes from CIFAR-10's size and high-dimensional image representation. With 60,000 images converted into feature vectors, repeated full-dataset memory accesses during each iteration create significant global memory bandwidth pressure, making the assignment step potentially memory-bound and limiting GPU scalability (*CodeSignal Learn*).

2.2 Mini-Batch K-Means Algorithm

The mini-batch K-means algorithm is a stochastic approximation of standard K-means designed to reduce computational cost and improve scalability on large datasets. Instead of processing the full dataset in each iteration, the algorithm randomly samples a small subset (mini-batch) of data points at each step. For each mini-batch, data points are assigned to the nearest centroid using the same Euclidean distance calculation as in the standard method. However, rather than recomputing centroids from the full dataset, centroid updates are performed incrementally using a learning-rate-based update rule that adjusts centroid positions based on the newly observed samples.

In this implementation, mini-batching significantly reduces memory bandwidth requirements and allows more frequent updates with lower computational overhead per iteration. This makes it particularly well suited for large-scale datasets such as CIFAR-10 when represented as high-dimensional feature vectors. While this approach introduces some approximation error compared to full batch updates, it often achieves comparable clustering quality with substantially improved runtime performance.

In the mini-batch implementation used in this project, the main bottleneck shifts from full-dataset memory access (seen in the batch version) to the centroid update stage. Because centroids are updated after every small batch, the implementation requires frequent incremental updates that can involve atomic operations or reduction-style accumulation across threads. This introduces

synchronization overhead and limits parallel efficiency, since multiple threads may attempt to update the same centroid concurrently. As a result, even though each iteration is cheaper than in the batch version, the repeated update operations become the dominant performance constraint in the mini-batch approach (*GeeksforGeeks)*.

Implementation

The implementation consists of two algorithmic versions of K-means clustering: a standard (batch) version and a mini-batch approximation. Both versions follow the same structure of iterative assignment and centroid update, but differ significantly in how computation is distributed and how memory is accessed on the GPU. The goal of the design is to expose parallelism in the distance computation stage while minimizing global memory overhead caused by the high dimensionality of CIFAR-10 feature vectors (3072 dimensions per image).

For both implementations, the dataset is stored as a flat, contiguous array of floating-point values in row-major order. This decision ensures coalesced memory access when threads read feature vectors, which is critical for GPU performance given that each distance computation requires scanning all 3072 dimensions. Centroids are also stored in a flat structure to simplify indexing and improve memory locality during repeated comparisons.

3.1 Standard (Batch) Implementation

In the standard implementation (GPU version), each CUDA thread is assigned one data point and is responsible for computing its distance to all centroids. This design was chosen as each point's assignment is independent. During the assignment step, threads compute squared Euclidean distance across all K centroids and store the index of the closest cluster. This stage is the most computationally expensive part of the algorithm and benefits most from parallel execution.

To reduce global memory bandwidth pressure, centroid values are loaded into shared memory in small tiles (controlled by TILE_K). Each tile is reused by all threads within a block, reducing redundant global memory reads when computing distances. Synchronization barriers (__syncthreads()) are used to ensure all threads have safely loaded centroid data before computation begins.

After assignment, centroid updates are performed using atomic operations on global memory. Each thread contributes to the sum of its assigned cluster, and cluster counts are incremented atomically. This design was chosen over block-level reductions to avoid complex inter-thread coordination and to maintain scalability across arbitrary dataset sizes, which proved to not scale well as the data size increased. While this introduces some contention, it simplifies the update logic and remains efficient given the relatively small number of clusters (K = 10).

3.2 Mini-Batch Implementation

The mini-batch implementation modifies the execution model by operating on randomly sampled subsets of the dataset rather than the full dataset each iteration. In this design, each iteration selects a random contiguous batch of data points, and the kernel processes only this subset. Each thread again handles one data point within the mini-batch, but overall workload per iteration is significantly reduced.

Instead of recomputing centroids from all assigned points, centroid updates are performed incrementally using a learning-rate-based rule. Each cluster is updated using a weighted combination of its previous value and the current mini-batch mean. This reduces both computation and memory bandwidth requirements, since full dataset reductions are no longer required.

To support this design, cluster statistics (sums and counts) are still accumulated using atomic operations, but only over the mini-batch subset. This reduces contention compared to full-batch execution, but introduces more frequent updates over time. As a result, computation shifts away from heavy per-iteration processing and toward more frequent but lightweight updates.

3.3 Implementation Tradeoffs

Across both implementations, the decision was made to keep the one thread per data point mapping for the assignment step, which maximizes parallel throughput and avoids complex load balancing. The tradeoff is that centroid updates require atomic synchronization, which was an anticipated bottleneck for scaling studies.

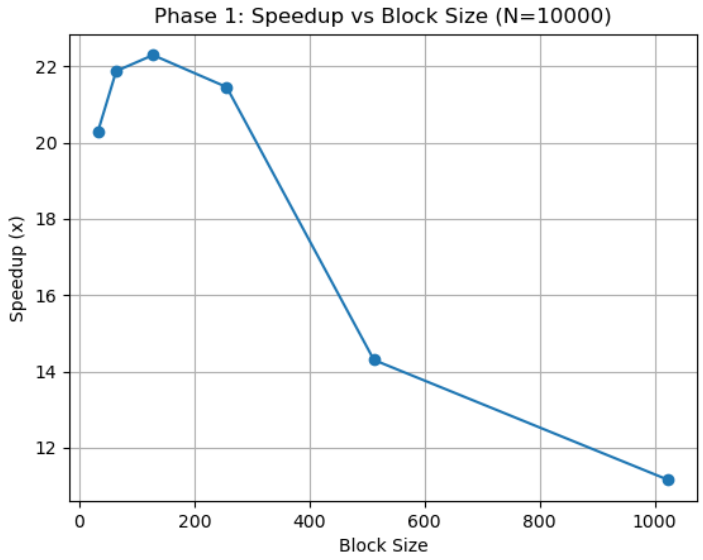
Shared memory tiling is used to reduce global memory traffic during centroid comparisons, while atomic updates ensure correctness without requiring block-wide reductions. The shift to mini-batching was introduced to reduce the dominant cost of full-dataset passes in the standard algorithm, specifically the repeated O(N) assignment and update steps over high-dimensional CIFAR-10 data. By operating on smaller, randomly sampled subsets, the algorithm reduces per-iteration computation and memory bandwidth pressure, improving overall runtime scalability.

Unfortunately, this change does not fully eliminate thread contention because centroid updates still occur at a shared global level (and was not able to be fully eliminated for the course of this project). Even though each iteration processes fewer points, threads within a mini-batch can still simultaneously update the same centroids, especially given the limited number of clusters (K = 10). As a result, atomic operations remain necessary to preserve correctness. The mini-batch approach therefore shifts the performance tradeoff from heavy global memory usage toward more frequent but smaller-scale synchronization events, rather than removing contention entirely.

Results

4.1 Speedup Performance Analysis

4.1.1 Phase 1: Block Size Scaling (N = 10,000)



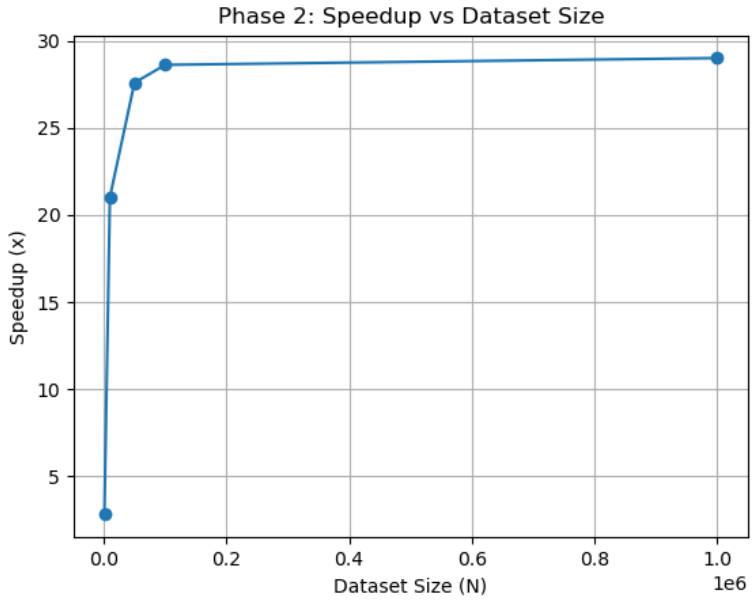
In this phase, performance is evaluated as a function of CUDA block size while keeping the workload fixed. The results show that speedup increases from 20.28x (block size 32) to a peak of 22.30x (block size 128), after which performance degrades significantly to 11.15x at block size 1024.

I believe this behavior is influenced by the kernel design and memory access patterns in the implementation. The kernel assigns one thread per data point, and each thread performs a full O(K × DIM) distance computation (where K = number clusters and DIM = dimension of each point in the dataset). With DIM = 3072, each thread executes a large number of arithmetic operations and repeatedly accesses centroid data.

At moderate block sizes (64–128), the GPU achieves good occupancy and warp scheduling efficiency, allowing latency to be hidden effectively. Additionally, the use of shared memory tiling (TILE_K = 2) improves reuse of centroid data across threads within a block, reducing global memory traffic.

However, at larger block sizes (512–1024), performance drops due to several interacting factors. First, shared memory usage per block increases, limiting the number of active blocks per SM and reducing occupancy. Second, the kernel uses __syncthreads() between tile loads, which introduces higher synchronization overhead as block size grows. Third, larger blocks amplify atomic contention in update_cluster_statistics, where many threads concurrently update the same cluster sums and counts. These effects collectively reduce parallel efficiency, explaining the observed decline in speedup.

4.1.2 Phase 2: Standard Implementation Grid Scaling (Block Size = 256)



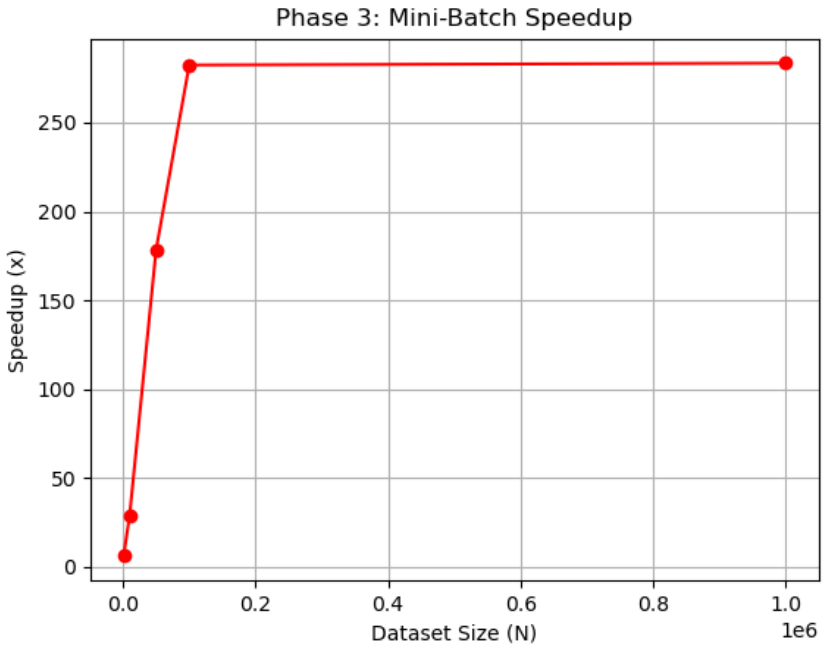
This phase evaluates how performance scales with dataset size. The results show that speedup increases from 2.81x at N = 1,000 to approximately 29x at N = 1,000,000, where the speedup seems to plateau.

This trend reflects the transition from underutilization to full GPU saturation. For small datasets, the number of threads (N) is not enough to fully occupy the GPU, resulting in low throughput. As N increases, more threads are launched, enabling better utilization of available compute resources and improved latency hiding.

However, as the dataset grows, performance gains begin to level off due to inherent bottlenecks in the design. The primary limitation is atomic contention during centroid updates. Each thread contributes to global cluster_sums and cluster_counts using atomicAdd, and as N increases, the number of concurrent updates grows significantly. This leads to partial serialization of updates, limiting further speedup. Additionally, the algorithm remains memory-bandwidth bound, as each thread repeatedly reads high-dimensional data (3072 floats) and centroid values from global memory, even with shared memory tiling.

Thus, while the implementation scales well initially, the plateau in speedup reflects the combined impact of atomic synchronization and memory bandwidth constraints.

4.1.3 Phase 3: Mini-Batch Scaling (Block Size = 256)



The mini-batch implementation demonstrates a dramatic increase in performance, with speedup rising from 6.22x at N = 1,000 to over 283x at N = 1,000,000. This behavior differs significantly from the standard implementation due to an algorithmic change outlined previously.

Instead of processing all N data points per iteration, the mini-batch version processes only a subset (batch_size = N / 10). This reduces the per-iteration computational complexity from O(N × K × DIM) to O(batch_size × K × DIM), leading to substantially lower GPU execution time.

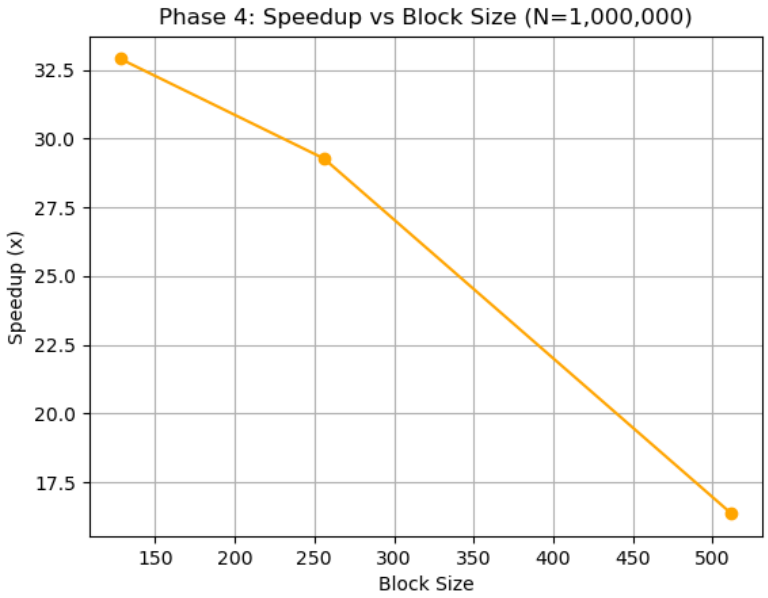
In the implementation, this is achieved by launching the same kernel (kmeans_minibatch_kernel) but on a randomly selected subset of the dataset, combined with a learning-rate-based centroid update. Because fewer threads are active per iteration, atomic contention is significantly reduced, and memory traffic is also lower.

However, I was unable to completely eliminate all thread contention. Threads within the batch still update shared cluster statistics using atomicAdd, meaning that moments can still occur when many points map to the same cluster. The key improvement of this implementation is that the frequency and intensity of contention are reduced, not removed.

Additionally, the mini-batch approach introduces stochastic updates, which trade deterministic convergence for speed. The learning rate smooths centroid updates over time, allowing the algorithm to converge despite noisy gradient estimates.

Overall, the large speedups observed in this phase are primarily driven by reduced workload per iteration and lower synchronization overhead, rather than purely improved hardware utilization.

4.1.4 Phase 4: Extreme Scaling (N = 1,000,000)



This phase evaluates performance at maximum workload to understand how block size impacts efficiency under heavy load. The results show that block size 128 achieves the highest speedup (32.90x), outperforming both 256 (29.27x) and 512 (16.35x).

This trend reflects the balance between occupancy, synchronization, and memory usage in the implementation. At extreme scale, block size 256 introduces more contention and synchronization overhead, while block size 128 allows more thread blocks to run concurrently across the GPU, improving overall utilization and latency hiding. This is especially important given the use of shared memory tiling and frequent __syncthreads() calls, where smaller blocks reduce synchronization costs.

In contrast, larger block sizes increase atomic contention and shared memory pressure, limiting scalability. Overall, performance at this scale is constrained less by parallelism and more by contention and resource limits, making block size a critical factor.

Future Improvements

While the current implementation demonstrates significant performance improvements from GPU acceleration and algorithmic approximation, several areas could further enhance efficiency, scalability, and numerical stability.

One major improvement would be replacing atomic-based centroid updates with a hierarchical reduction strategy. In the current implementation, atomicAdd operations are used to accumulate cluster statistics, which can create contention when many threads update the same centroid. A more optimized approach would be to first perform block-level shared memory reductions,

followed by a single global write per block. This would significantly reduce atomic pressure and better align with CUDA's memory hierarchy.

The centroid update rule in mini-batch K-means could also be improved. Currently, updates use a fixed learning rate, but a more robust approach would use an adaptive learning rate schedule based on the number of updates per centroid (while I do think an algorithmic optimization like this might creep outside the scope of this class). This might improve convergence consistency, especially for uneven cluster assignments.

Conclusion

These experiments demonstrate that K-means clustering can be significantly accelerated using GPU parallelization, particularly when combined with memory hierarchy optimizations and algorithmic approximations. The CUDA implementation achieves substantial speedups over the CPU baseline by mapping distance computations to parallel threads, reducing global memory overhead through shared memory tiling, and efficiently distributing workload across high-dimensional data.

Across the experiments, performance was strongly influenced by both hardware configuration (block size and workload scaling) and algorithmic design choices. The standard GPU implementation shows strong scaling behavior as dataset size increases, reaching saturation at large N due to atomic update contention and memory bandwidth limitations. This leads to a noticeable plateau in speedup (~28–29x), where increasing dataset size no longer produces proportional performance gains. This plateau indicates that the system transitions from being compute-bound to being limited by shared resource contention and global memory bandwidth, particularly during centroid update operations and repeated high-dimensional memory accesses.

The mini-batch version further improves performance by reducing the effective workload per iteration and limiting the number of concurrent updates to shared centroids. This leads to dramatic speedups, but introduces a tradeoff between computational efficiency and deterministic convergence. Importantly, the results show that while mini-batching reduces contention and memory pressure, it does not fully eliminate synchronization overhead due to shared centroid updates.

Block size experiments further reinforce the importance of balancing occupancy, synchronization cost, and memory usage. Smaller block sizes were more effective at extreme scale, where reduced contention and improved scheduling outweighed the benefits of larger parallel groups.

Overall, the project illustrates that GPU acceleration of K-means is most effective when both algorithmic structure and hardware-level execution details are considered together. The largest gains come not only from parallelizing computation, but from carefully managing memory

access patterns, reducing synchronization overhead, and introducing controlled approximations such as mini-batching.

References

"Enhancing Machine Learning Expertise: Mini-Batch K-Means Clustering Explained." *CodeSignal Learn*, codesignal.com/learn/courses/unsupervised-learning-and-clustering/lessons/enhancing-machine-learning-expertise-mini-batch-k-means-clustering-explained. Accessed 3 May 2026.

"K Means Clustering – Introduction." *GeeksforGeeks*, GeeksforGeeks, 10 Nov. 2025, www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/.